

David Kan

Part A: Program Structure and IPC

Basically, the program takes a file and splits it into equal chunks, one for each child process.

First the parent opens the file and finds the total size using `fseek` and `ftell`, then calculates the child's byte offset and chunksize by dividing the file size by the num of requested processes. The last child gets any leftover bytes from the integer division so the entire file gets read.

Every child process has a dedicated pipe for communication. Prior to forking each child, the parent calls `pipe()`, creating a pair of file descriptors and stores them into a `plist_t` struct which also has the child's PID, offset, chunksize, and status. After the `fork()`, the child closes the read end of the pipe as it only writes, and the parent closes the write end as it only reads.

In the child process, the file is opened separately with `fopen()`, and `word_count()` is called with the child's assigned offset and size. The `word_count` function uses `fseek` to go to the right position, then starts reading char by character counting the lines, words, and characters in its assigned chunk.

When counting is done, the child writes the `count_t` struct (with `linecount`, `wordcount`, and `charcount`) to the pipe's write end using the `write()` syscall, then closes the descriptors and exits.

The parent waits for the children by calling `waitpid(-1, &status, 0)` in a loop, which blocks until any child terminates. When a child exits normally, the parent identifies the job it belonged to by matching the returned PID with the `plist` array and reads the `count_t` result from the child's pipe read end, adding the values to the total. Once all the children have finished and their results have been collected, the parent prints the totals.

Part B: Crash Handling

The parent detects a crash by looking at the status value returned by `waitpid()`. It checks `WIFEXITED(status)` to see if the child exited normally and `WIFSIGNALED(status)` to see if it was killed by a signal. If `WIFSIGNALED` returns true, the parent knows the child crashed and logs which signal caused the termination using `WTERMSIG(status)`. When this happens, the parent closes the old pipe's read end since no data was written to it, then recalls the `launch_child` function creating a fresh pipe and forks a new child process for that same job. As the old `plist_t` struct still has the original offset and chunksize, the replacement child processes the exact same portion of the file. If the new child also crashes, the cycle repeats.

The parent also tracks how many jobs are still pending which a job is marked done once a child exits normally and the parent successfully reads the result from the pipe. The loop continues

until all of the jobs are complete. In order to prevent infinite loops in extreme cases (such as `crashrate == 100`), the program caps the total num of forks to `MAX_FORK(1000)`.